08 Triggers & Transactions

1: Trigger - Log every deletion in the Orders table

Before Trigger:

```
SELECT * FROM Orders;
SELECT * FROM Order_History;
```

OrderID	CustomerName	OrderDate	Amount
101	Aadhi	2026-05-14	500
102	Bala	2026-02-14	600
103	Christ	2026-02-12	750
104	David	2026-1-23	1050
105	Eswari	2025-12-23	1024

OrderID	CustomerName	OrderDate	Amount	DeletedAt

Trigger For deleting row in table:

```
DELIMITER $$
CREATE TRIGGER LogOrderDeletion
AFTER DELETE ON Orders
FOR EACH ROW
BEGIN
    INSERT INTO Order_History (OrderID, CustomerName, OrderDate, Amount, DeletedAt)
    VALUES (OLD.OrderID, OLD.CustomerName, OLD.OrderDate, OLD.Amount, NOW());
END $$
DELIMITER ;
```

Dleting Row:

```
DELETE FROM Orders WHERE OrderID = 101;
```

Result Of Trigger:

```
SELECT * FROM Orders;
SELECT * FROM Order_History;
```

OrderID	CustomerName	OrderDate	Amount
102	Bala	2026-02-14	600.00
103	Christ	2026-02-12	750.00
104	David	2026-01-23	1050.00
105	Eswari	2025-12-23	1024.00

OrderID	CustomerName	OrderDate	Amount	DeletedAt
101	Aadhi	2026-05-14	500.00	2026-04-04 11:05:28

2: DCL Commands - Grant reporting access to Junior analysts

Created Junior Analyst user and view created user:

```
CREATE USER 'junior_analyst'@'localhost' IDENTIFIED BY '[redacted]';
SELECT User, Host FROM mysql.user;
```

User	Host
junior_analyst	localhost
mysql.infoschema	localhost
mysql.session	localhost
mysql.sys	localhost
root	localhost

Granting access to created Junior Analyst

```
GRANT SELECT ON employee.* TO 'junior_analyst'@'localhost';
```

Revoke access to created Junior Analyst

```
REVOKE SELECT ON employee.* FROM 'junior_analyst'@'localhost';
```

3: TCL Commands - During a bank transfer, ensure atomicity using COMMIT, ROLLBACK, SAVEPOINT

Created DB and table:

```
CREATE DATABASE BankDB;
USE BankDB;

CREATE TABLE Accounts (AccountID INT PRIMARY KEY,AccountHolder VARCHAR(50),Balance DECIMAL(10,2));
INSERT INTO Accounts (AccountID, AccountHolder, Balance) VALUES(101, 'Aadhi', 5000.00),
(102, 'Bala', 3000.00),(103, 'Christ', 7000.00);
SELECT * FROM Accounts;
START TRANSACTION;
```

AccountID	AccountHolder	Balance
101	Aadhi	5000.00
102	Bala	3000.00
103	Christ	7000.00

Commit:

```
UPDATE Accounts SET Balance = Balance - 500
WHERE AccountID = 101;

UPDATE Accounts SET Balance = Balance + 500
WHERE AccountID = 102;

COMMIT;
```

AccountID	AccountHolder	Balance
101	Aadhi	4500.00
102	Bala	3500.00
103	Christ	7000.00

Rollback:

```
START TRANSACTION;

UPDATE Accounts SET Balance = Balance - 1000
WHERE AccountID = 101;

UPDATE Accounts SET Balance = Balance + 1000
WHERE AccountID = 103;
```

AccountID	AccountHolder	Balance
101	Aadhi	3500.00
102	Bala	3500.00
103	Christ	8000.00

```
ROLLBACK;
```

Result after RollBack:

AccountID	AccountHolder	Balance
101	Aadhi	4500.00
102	Bala	3500.00
103	Christ	7000.00

SavePoint:

```
START TRANSACTION;

UPDATE Accounts SET Balance = Balance - 200
WHERE AccountID = 101;
```

AccountID	AccountHolder	Balance
101	Aadhi	4300.00
102	Bala	3500.00
103	Christ	7000.00

Creating Savepoint:

```
SAVEPOINT Step1;

UPDATE Accounts SET Balance = Balance + 200
WHERE AccountID = 102;
```

AccountID	AccountHolder	Balance
101	Aadhi	4300.00
102	Bala	3700.00
103	Christ	7000.00

Rollback to savepoint:

```
ROLLBACK TO Step1;
```

AccountID	AccountHolder	Balance
101	Aadhi	4300.00
102	Bala	3500.00
103	Christ	7000.00